SCALER | Priyansh Saxena

RAG Pipeline Design and Vector Databases

Retrieval-Augmented Generation ▪ Chunking ▪ Vector DBs ▪ Hybrid Search ▪ Evaluation

Priyansh Saxena

Scaler School of Technology | ML System Design and LLMOps

Week 4 — Session 1/2

ROADMAP

Agenda

1. Why RAG?
2. Chunking and Embedding
3. Vector Databases
4. Hybrid Search and Reranking
5. Evaluation and Failure Modes

TOC

1

Why RAG?

1 The Problem with LLMs Out of the Box

What LLMs know

- Parametric memory: knowledge baked into weights during training
- Strong reasoning, summarisation, code generation
- Training cutoff: GPT-4 knows nothing after a fixed date
- Context window: limited — cannot fit your entire knowledge base

What LLMs fail at

- Facts from yesterday's news
- Your company's internal documents
- Precise citations (it invents them)
- Recent product releases, regulation changes
- Confidential data not in training corpus

The hallucination problem

An LLM asked about facts it was not trained on does not say "I don't know." It **confidently fabricates** a plausible-sounding answer. This is a fundamental property of next-token prediction.

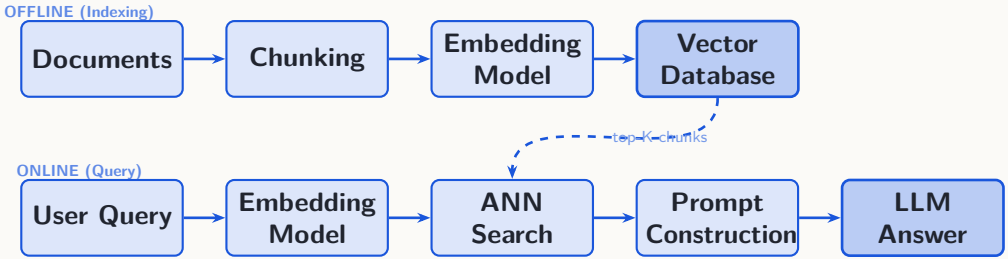
1 RAG vs. Fine-Tuning vs. Prompt Engineering

	Prompt Engineering	RAG	Fine-Tuning
Speed	Minutes	Hours (index build)	Hours to days
Cost	Cheapest	Moderate	Most expensive
Knowledge updates	Manual edit	Re-index docs	Full retrain
Best for	Task format, tone within context window	Frequently changing knowledge, large corpora, citations	Specific style, tone, or task format
Adds new facts?	No	**Yes**	Partially (unreliable)

Rule of thumb

Fine-tuning does **not** reliably teach new facts — it teaches the model *how to behave*. RAG is for *what to know*. Use both when you need both.

1 RAG Architecture Overview



Key constraint: the embedding model used at index time **must be the same** as at query time.

2

Chunking and Embedding

2 Chunking Strategies

Fixed-size chunking

- Split every N tokens with M tokens overlap
- Simple, fast, deterministic
- Problem: splits mid-sentence, mid-concept
- Rule of thumb: 512 tokens, 50 overlap as starting point

Sentence / paragraph chunking

- Split on natural language boundaries (., \n\n)
- Better semantic coherence per chunk
- Variable chunk sizes complicate batch processing

Recursive chunking

- Try splitting on: paragraphs → sentences → words
- Hierarchical fallback — chunk size is bounded
- LangChain's `RecursiveCharacterTextSplitter`

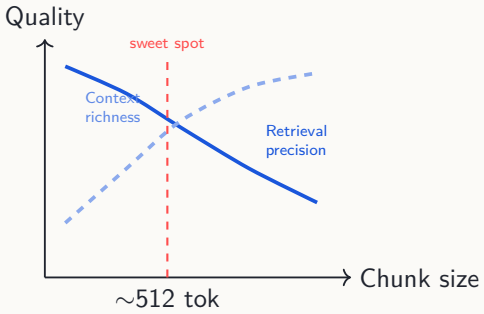
Semantic chunking

- Embed sentences, find topic boundary where embedding similarity drops
- Best quality, most expensive
- Good for long, heterogeneous documents

Late chunking (2024)

Embed the full document first, then split the embeddings. Preserves long-range context in each chunk's vector.

2 The Chunk Size Trade-off



Small chunks

+ Precise retrieval — matches exactly what is asked
– Loses surrounding context
– More vectors to store and search

Large chunks

+ Rich context per retrieved passage
– Noisy retrieval — matches on tangential content
– Fewer tokens left in context window for other chunks

Iterative tuning on your specific dataset beats any rule of thumb.

2 Embedding Models

Model	Key facts
OpenAI text-embedding-3-small	1536 dims, $0.02/1M tokens, strong quality, easy API
E5-large-instruct (Microsoft)	Open source, 1024 dims, instruction-tuned, competitive with OpenAI
BGE-M3 (BAAI)	Multilingual; dense + sparse + multi-vector; excellent for hybrid search

The cardinal rule

The embedding model at **index time** must be **identical** to the model at **query time**. A version mismatch produces vectors in different spaces — cosine similarity becomes meaningless and **retrieval silently degrades to random**.

Dimensions trade-off

- More dims ⇒ richer representation
- More dims ⇒ more storage, slower ANN search
- OpenAI v3 supports *dimension truncation* (Matryoshka)

2 Quiz 1

Q1 — Chunking Strategy

You are building a RAG system over a collection of academic papers (PDFs, 10–30 pages each). Each paper has: abstract, introduction, methods, results, conclusion.

Which chunking strategy would you use and why? What chunk size would you start with?

Bonus: How would your answer change if users ask highly specific questions about equations in the Methods section?

Q2 — Embedding Model Version Mismatch

Your team indexed 500,000 documents using `text-embedding-ada-002`. Six months later, a new team member updates the query code to use `text-embedding-3-small`. No errors are thrown. Retrieval quality drops from 85% to 23%.

Why does this happen? How would you detect it before users complain?

2 Quiz 1 — Answers

A1 — Chunking Strategy

Recommended: section-aware recursive chunking. Split first by section boundary (Abstract, Introduction, Methods...), then recursively by paragraph within each section. Preserves semantic coherence — a Methods chunk won't bleed into Results.

Starting chunk size: 512 tokens, 64-token overlap. Academic writing is dense; shorter chunks lose context, longer chunks dilute precision.

Bonus — equations in Methods: smaller chunks (256 tokens) for Methods specifically, and add the section title as metadata. Equation questions need precise retrieval, not broad context.

A2 — Embedding Model Version Mismatch

Why it happens: `ada-002` and `text-embedding-3-small` produce vectors in different semantic spaces. Cosine similarity between vectors from different models measures distance in the wrong space — retrieval degrades to near-random.

How to detect before users complain: (1) monitor the **distribution of top-1 cosine similarity** over time — a sudden drop is a red flag; (2) run a **retrieval regression test** on a golden set of 50 query–answer pairs nightly; alert if Recall@5 drops $> 5\%$.

3

Vector Databases

3 What a Vector Database Does

Core operations

- **Upsert**: store (id, vector, metadata)
- **Delete**: remove by id
- **Similarity search**: given query vector, return top-K most similar stored vectors
- **Metadata filtering**: "find top-K chunks *where source='legal-docs'*"
- **Batch upsert**: for bulk indexing pipelines

Similarity metrics supported

- Cosine similarity (most common for text)
- Dot product (when vectors are normalised, $\equiv$ cosine)
- Euclidean (L_2) distance

Why not just use PostgreSQL?

A full exhaustive scan (`SELECT * ORDER BY distance`) is $O(n)$ per query. At 10M vectors this takes seconds. Vector DBs use **approximate nearest neighbour (ANN)** algorithms to return results in milliseconds. You trade a small recall loss ($\sim$1–5%) for 100$\times$ speed.

pgvector

PostgreSQL extension that adds vector search. Slower ANN than dedicated DBs (uses IVFFlat or HNSW). Great if you already run Postgres and want minimal new infra.

3 ANN Algorithms: HNSW vs. IVF

HNSW (Hierarchical Navigable Small World)

- Graph-based: nodes are vectors, edges connect near neighbours
- Multi-layer graph — coarse to fine
- Query: greedy traversal from entry point
- Complexity: $O(\log n)$ per query
- High recall at low latency
- Memory intensive (graph stored in RAM)
- Used by: Qdrant, Weaviate, pgvector, Chroma

IVF (Inverted File Index)

- Cluster-based: train k-means centroids offline
- Index each vector to its nearest centroid's bucket
- Query: search only the top-n_{probe} buckets
- Lower memory than HNSW
- Recall depends on n_{probe} (tunable)
- Requires training step on data
- Used by: Faiss (Meta), Pinecone internals

Trade-off

HNSW: better recall, more RAM. IVF: less RAM, needs tuning. HNSW is the default choice for most use cases.

3 Vector Database Comparison

DB	Type	Strengths	Best for
Chroma	Open source	Lightweight, zero setup, Python-native	Prototypes, local dev, small scale
pgvector	Postgres ext.	No new infra if you run Postgres, SQL interface	Teams already on Postgres; simple filtering
Qdrant	Open source / cloud	Rust-based, very fast, excellent payload filtering	Production with complex metadata filters
Weaviate	Open source / managed	Hybrid search native, vectorisation modules, GraphQL	Production needing hybrid search out-of-box
Pinecone	Fully managed	Serverless, zero-ops, easy scaling	Teams wanting no infra overhead; quick start to prod

Decision rule

Prototype → **Chroma**. Existing Postgres stack → **pgvector**. Need metadata filtering → **Qdrant**. Need hybrid search → **Weaviate**. Zero ops at scale → **Pinecone**.

5-Minute Break

Your vector database is indexed. Your brain needs a reindex too.
Step away, grab a drink, and let the embeddings settle.

See you in five.

4

Hybrid Search and Reranking

4 Dense vs. Sparse Retrieval

Dense retrieval (embedding search)

- Represents query and documents as dense vectors
- Captures **semantic similarity**
- Finds: "What is the capital of France?" ↔ "Paris is the capital" *even with no keyword overlap*
- Fails on: exact product codes, version numbers, names

Sparse retrieval (BM25 / TF-IDF)

- Represents documents as sparse term-frequency vectors
- Captures **exact keyword matches**
- Great for: "Python 3.11 release notes", "CVE-2024-1234", product IDs
- Misses: paraphrase, synonyms, cross-lingual

They are complementary

- Dense: strong on semantics, weak on keywords
- Sparse: strong on keywords, weak on semantics
- Neither dominates in all cases

Production rule

Always use hybrid search in production. Pure dense misses exact keyword matches. Pure sparse misses semantic matches. Hybrid covers both failure modes.

4 Hybrid Search: Combining Dense + Sparse

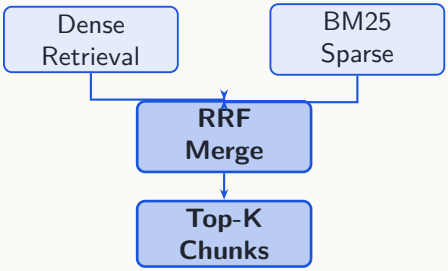
Reciprocal Rank Fusion (RRF)

- Dense retrieval returns ranked list R_d
- Sparse retrieval (BM25) returns ranked list R_s
- RRF score for document d:

$$\text{RRF}(d) = \sum_{r \in \{R_d, R_s\}} \frac{1}{k + \text{rank}_r(d)}$$

where $k = 60$ is a smoothing constant

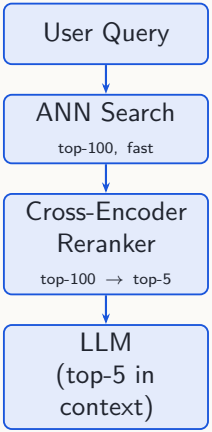
- Merge both lists by RRF score, take top-K
- No training required, robust to score scale differences
- Supported natively: Weaviate, Elasticsearch, Qdrant



4 Reranking: Precision After Recall

Why reranking?

- ANN search optimises for **speed**: top-100 in milliseconds
- But ANN uses a **bi-encoder**: query and document encoded separately
- A **cross-encoder** takes (query, document) as a single input
- Cross-encoders are far more accurate — full attention across both
- Too slow to use at retrieval time (no pre-indexing possible)
- Solution: ANN retrieves top-100 (fast), cross-encoder reranks top-100 (accurate), pass top-5 to LLM



Models: ms-marco-MiniLM-L-12-v2, Cohere Rerank API

5

Evaluation and Failure Modes

5 RAGAS: Evaluating RAG Pipelines

Four core metrics (RAGAS framework)

- **Faithfulness**: does the LLM answer use *only* the retrieved context? Score 0–1. Catches hallucinations.
- **Answer Relevancy**: is the answer relevant to the user's question?
- **Context Precision**: of the retrieved chunks, what fraction were actually relevant to the question?
- **Context Recall**: did the retrieval return all chunks needed to answer the question?

Why you need all four

- High faithfulness + low context recall $\Rightarrow$ model honest, but retrieval misses key info
- High context recall + low faithfulness $\Rightarrow$ right docs retrieved, but LLM ignores them and hallucinates
- High precision + low recall $\Rightarrow$ retrieved chunks are relevant but incomplete

Tools: `ragas` library, TruLens, DeepEval

5 5 Common RAG Failure Modes

Failure	What happens	How to detect
Retrieval failure	Right info exists in corpus but not returned (wrong chunking, wrong embedding, low ANN recall)	Measure context recall; run retrieval unit tests
Generation failure	Correct chunks retrieved but LLM ignores them and hallucinates anyway	Measure faithfulness score; citation verification
Context overflow	Too many chunks retrieved; LLM loses info in middle ("lost in the middle" effect)	Limit top-K; rerank to top-5; chunk quality over quantity
Model version mismatch	Embedding model updated at query time but index uses old model; vectors in different spaces	Lock embedding model version; alert on cosine score distribution shift
Stale index	Document updated but old chunk still indexed; LLM answers with outdated information	Track document update timestamps; trigger re-indexing on update

5 Guardrails and Hallucination Mitigation

Input guardrails

- **Prompt injection detection**: "Ignore all previous instructions and..."
- **PII filtering**: redact personal data before it enters the LLM
- **Topic scoping**: reject queries outside the system's domain
- **Confidence threshold**: if max similarity score $< \theta$, respond with "I don't have information on this"

Output guardrails

- **Faithfulness check**: verify the answer is grounded in retrieved context (NLI-based)
- **Citation enforcement**: require LLM to cite chunk id; verify citation exists in retrieved set
- **PII in output**: scan generated text for leaked personal data

Confidence threshold strategy

If the top retrieved chunk has similarity < 0.7 (cosine), the document does not exist in your corpus. Saying "I don't know" is correct and maintains trust.

5 Quiz 2

Q1 — Failure Mode Diagnosis

Your RAG system answers user questions about your product documentation. You notice that when users ask about a feature updated 2 weeks ago, the system gives the old (wrong) answer confidently, despite the docs being updated.

Which failure mode is this? What is the fix?

Q2 — Evaluation Design

A RAG system over legal contracts achieves faithfulness $= 0.95$ and answer relevancy $= 0.91$. The team declares success. However, users still complain that "the system doesn't find the relevant clause."

Which RAGAS metric was the team *not* measuring? What does a low score on that metric imply about the pipeline?

5 Quiz 2 — Answers

A1 — Failure Mode Diagnosis

Failure mode: Stale index. The document was updated but the old chunk was not removed and re-indexed. The vector DB still returns the outdated chunk with high similarity, so the LLM confidently answers with old information.

Fix: (1) track a `last_updated` timestamp per source document; (2) on update, delete all chunks with that document's ID and re-index the new version; (3) add a metadata filter to exclude chunks older than the last-known-good timestamp.

A2 — Evaluation Design

Missing metric: Context Recall. Faithfulness 0.95 means the LLM answers from retrieved context (not hallucinating); answer relevancy 0.91 means it is on-topic. But users complain because the *right clause is never retrieved* — a retrieval failure, not a generation failure.

Low context recall implies: the relevant clause is (a) not in the index, (b) chunked so it splits across two chunks that neither score high enough, or (c) semantically distant from the query wording (dense retrieval failing on legal terminology — a hybrid search fix).

5 What We Covered Today

§1 Why RAG?

- LLM limitations: cutoff, hallucination, no citations
- RAG vs. fine-tuning vs. prompt engineering
- Full RAG architecture: indexing + query pipeline

§2 Chunking and Embedding

- Fixed, sentence, recursive, semantic chunking
- Chunk size trade-off: precision vs. context
- Embedding models: OpenAI, E5, BGE
- Cardinal rule: same model at index and query time

§3 Vector Databases

- Core operations: upsert, search, filter
- ANN: HNSW vs. IVF trade-offs
- DB comparison: Chroma, pgvector, Qdrant, Weaviate, Pinecone

§4 Hybrid Search & Evaluation

- Dense + sparse = hybrid; RRF merging
- Cross-encoder reranking pipeline
- RAGAS: faithfulness, relevancy, precision, recall
- 5 failure modes and guardrails

Thank You!

Your LLM hallucinates. But with RAG, at least it hallucinates with receipts.

Priyansh Saxena

All materials at:

https://www.priyanshsaxena.com/sst-ml-system-design

